

# APCS 初級

## Python 考前總複習

程式識讀 × 程式實作

程式識讀 30 題四選一，每題 5 分，共 150 分，75 分鐘

程式實作 2 題，合計 100 分，75 分鐘

核心能力 追蹤執行流程、條件 / 迴圈 / 遞迴、陣列操作

依據官方題本範例整理

# 目錄

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>1</b> | <b>基礎語法速覽</b>                          | <b>3</b>  |
| 1.1      | 資料型別與運算子 . . . . .                     | 3         |
| 1.2      | 輸入與輸出 . . . . .                        | 3         |
| <b>2</b> | <b>條件判斷</b>                            | <b>4</b>  |
| 2.1      | if / elif / else 結構 . . . . .          | 4         |
| 2.2      | 條件順序錯誤 (題目 17) . . . . .               | 4         |
| 2.3      | 獨立 if 的追蹤 (題目 10) . . . . .            | 5         |
| <b>3</b> | <b>迴圈</b>                              | <b>6</b>  |
| 3.1      | for 迴圈與 range() . . . . .              | 6         |
| 3.2      | while 迴圈 . . . . .                     | 6         |
| 3.3      | 巢狀迴圈與圖形 (題目 5、19) . . . . .            | 7         |
| 3.4      | 二維陣列巢狀迴圈 (題目 18) . . . . .             | 7         |
| <b>4</b> | <b>函式與遞迴</b>                           | <b>8</b>  |
| 4.1      | 函式基本概念 . . . . .                       | 8         |
| 4.2      | 遞迴的兩個必要條件 . . . . .                    | 8         |
| 4.3      | 遞迴追蹤的方法 . . . . .                      | 9         |
| 4.4      | 費式數列 (題目 7、14) . . . . .               | 9         |
| 4.5      | GCD 輾轉除法 (題目 9、23) . . . . .           | 10        |
| 4.6      | 互相遞迴 (題目 8、13) . . . . .               | 10        |
| 4.7      | 遞迴冪次 (題目 21) . . . . .                 | 11        |
| <b>5</b> | <b>串列 (List)</b>                       | <b>11</b> |
| 5.1      | 基本操作 . . . . .                         | 11        |
| 5.2      | 串列初始化 . . . . .                        | 12        |
| 5.3      | 同時賦值 (題目 15) . . . . .                 | 12        |
| 5.4      | 索引偏移 (題目 25) . . . . .                 | 12        |
| 5.5      | 搜尋演算法 (題目 6、22、29) . . . . .           | 13        |
| 5.6      | 最大值、最小值追蹤 (題目 24、29) . . . . .         | 13        |
| <b>6</b> | <b>程式識讀技巧</b>                          | <b>14</b> |
| 6.1      | 執行流程追蹤四步驟 . . . . .                    | 14        |
| 6.2      | 死程式碼 (Dead Code, 題目 27) . . . . .      | 14        |
| 6.3      | 冗餘程式碼 (題目 20) . . . . .                | 14        |
| 6.4      | 填空題策略 (題目 2、3、9、12、16、19、23) . . . . . | 15        |

|          |                              |           |
|----------|------------------------------|-----------|
| <b>7</b> | <b>程式實作題攻略</b>               | <b>15</b> |
| 7.1      | 解題流程 . . . . .               | 15        |
| 7.2      | 實作題型一：七言對聯（2021/9） . . . . . | 15        |
| 7.3      | 實作題型二：裝飲料（2024/10） . . . . . | 17        |
| 7.4      | 實作題型三：遊戲選角（2024/1） . . . . . | 18        |
| <b>8</b> | <b>程式識讀完整題庫</b>              | <b>20</b> |
| <b>9</b> | <b>考前快速記憶表</b>               | <b>39</b> |
| 9.1      | 必背語法 . . . . .               | 39        |
| 9.2      | 識讀題分類速查 . . . . .            | 39        |
| 9.3      | 實作題得分策略 . . . . .            | 39        |
|          | <b>練習題解答</b>                 | <b>40</b> |

## 1 基礎語法速覽

### 1.1 資料型別與運算子

Python 常用型別：

| 型別    | 範例          | 說明   | 轉換函式     |
|-------|-------------|------|----------|
| int   | 42, -5      | 整數   | int(x)   |
| float | 3.14        | 浮點數  | float(x) |
| str   | 'hello'     | 字串   | str(x)   |
| bool  | True, False | 布林值  | bool(x)  |
| list  | [1, 2, 3]   | 可變序列 | list(x)  |

算術運算子

| 運算子        | 說明         | 範例             |
|------------|------------|----------------|
| +, -, *, / | 加減乘（浮點）除   | 7 / 2 = 3.5    |
| //         | 整數除法（向下取整） | 7 // 2 = 3     |
| %          | 取餘數        | 7 % 2 = 1      |
| **         | 次方         | 2 ** 10 = 1024 |

#### 常見錯誤

// 是向下取整（floor），負數時要小心：`-7 // 2 = -4`，不是 `-3`。APCS 題目的 `(low + high) // 2` 在非負整數時沒問題。

### 1.2 輸入與輸出

```

1 n = int(input())           # 讀一個整數
2 a, b = map(int, input().split()) # 同一行讀兩個整數
3 nums = list(map(int, input().split())) # 讀一行多個整數成串列
4
5 print(a, b)                # 輸出，預設空格分隔，自動換行
6 print(a, b, sep='')        # 兩數緊鄰，無空格
7 print('Hi!', end='')       # 不換行
8 print(f'答案是 {a + b}')    # f-string 格式化

```

#### 考試提示

實作題裡，輸出格式是評分關鍵。看清楚是空格分隔還是換行，輸出 `None`（字串）還是沒有輸出，一個字都不能錯。

## 2 條件判斷

### 2.1 if / elif / else 結構

```
if 條件 A:
    # 執行 A
elif 條件 B:
    # 執行 B (A 不成立才判斷)
else:
    # A、B 都不成立
```

#### 核心觀念

**if 與 elif 的根本差異**

`if ... elif ...` 是一組，只執行第一個成立的分支，其餘全跳過。  
兩個獨立的 `if` 則各自判斷，兩個都可能執行。

### 2.2 條件順序錯誤（題目 17）

題目問「評定等第的程式有幾個分數寫錯」：

```
1 if s >= 90:
2     print('A')
3 elif s >= 80:
4     print('B')
5 elif s > 60:           # 錯：應寫 s >= 70 才能涵蓋 70~79
6     print('D')         # 60~79 都跑進來印 D
7 elif s > 70:           # 這行永遠執行不到！
8     print('C')
9 else:
10    print('F')
```

兩個 bug：`elif s > 60` 寫成 `>` 而非 `>=`，且順序放在 C 之前。對照每個區間，看實際印出了什麼：

| 分數範圍   | 應印 | 實際印 | 原因                                   | 正確？   |
|--------|----|-----|--------------------------------------|-------|
| 90~100 | A  | A   | —                                    | ✓     |
| 80~89  | B  | B   | —                                    | ✓     |
| 70~79  | C  | D   | <code>s &gt; 60</code> 先攔截，印 D       | × ×10 |
| 61~69  | D  | D   | <code>s &gt; 60</code> 攔截，印 D        | ✓     |
| 60     | D  | F   | <code>s &gt; 60</code> 不含 60，落進 else | × ×1  |
| 0~59   | F  | F   | —                                    | ✓     |

錯誤共  $10 + 1 = 11$  個，答案 (B)。

## 常見錯誤

多層條件的地雷：

1. 範圍是否有遺漏或重疊 ( `>` vs `>=` )
2. 上面的 `elif` 先成立，下面的就不看
3. 邊界值（如恰好等於臨界點的那個數）最容易出錯

## 2.3 獨立 if 的追蹤（題目 10）

```

1 count = 10
2 if count > 0:
3     count = 11          # count = 11
4 if count > 10:          # 第二個獨立 if，仍會判斷
5     count = 12
6     if count % 3 == 4:  # 12 % 3 = 0，不等於 4
7         count = 1
8     else:
9         count = 0       # count = 0
10 elif count > 11:       # 這個 elif 屬於第二個 if，被跳過
11     count = 13
12 else:
13     count = 14
14 if count:              # count == 0 → False
15     count = 15
16 else:
17     count = 16          # count = 16
18 print(count)          # 輸出 16，答案 (D)

```

## 習題—條件追蹤

以下程式執行後，`x` 的值為何？

```

1 x = 5
2 if x > 3:
3     x = x + 2
4 if x > 6:
5     x = x * 2
6 elif x > 4:
7     x = x - 1
8 print(x)

```

## 3 迴圈

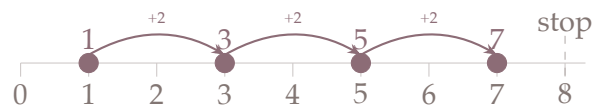
### 3.1 for 迴圈與 range()

```

1 for i in range(5):           # i = 0, 1, 2, 3, 4
2 for i in range(2, 7):        # i = 2, 3, 4, 5, 6
3 for i in range(10, 0, -1):   # i = 10, 9, 8, ..., 1
4 for i in range(0, 101, 5):   # i = 0, 5, 10, ..., 100 (共 21 個)

```

`range(1, 8, 2)` 視覺化



#### 題目 26：range 的元素數量

`range(0, 101, 5)` 含 21 個值（從 0 到 100），不是 20 個。

公式： $\left\lfloor \frac{\text{stop} - \text{start} - 1}{\text{step}} \right\rfloor + 1$ ，或直接數 0, 5, 10, ..., 100。

要印恰好 20 次，應改為 `range(0, 100, 5)` 或 `range(1, 101, 5)`。

#### 習題—range 計算

不用電腦，直接算出以下 `range()` 各有幾個元素：

- (a) `range(3, 12)`
- (b) `range(0, 20, 3)`
- (c) `range(10, 1, -2)`

### 3.2 while 迴圈

```

1 i = 0
2 while i < 10:
3     print(i)
4     i += 1           # 更新變數，確保能到達終止條件

```

解「填空 while 條件」的策略：先確定迴圈應該在什麼情況停下，再把停止條件取反得到「繼續執行」的條件。

#### 題目 2：填入 while 條件

目標輸出：[1][2][3][5][4][6]

外層 `for i in range(5)`，內層 `j` 從 0 開始，每次 `j = j + 2`，印出 `i + j`。

- `i=0`：沒有 `j` 值滿足條件，不印
- `i=1`：`j=0` 印 [1]

- `i=2` : `j=0` 印 `[2]` , `j=2` 印 `[4]` (但輸出只有 `[2][3]` )

對照輸出, `i=2` 應只印一次, 所以 `j < i` (即 `j < 2` ) 只允許 `j=0` 。

答案 (A) : 條件為 `j < i` 。

### 3.3 巢狀迴圈與圖形 (題目 5、19)

解題步驟：

1. 算每一列的空格數與符號數
2. 外層 `i` 為列號，推出空格數和符號數關於 `i` 的公式
3. 調整 `range()` 的 `start`、`stop`、`step`

題目 19，要印出倒三角形（每列 `6-2i` 個星號）：

```

1 # 目標輸出：
2 # * * * * * *      (i=0, 6 個星)
3 # * * * *          (i=1, 4 個星)
4 # * *              (i=2, 2 個星)
5 for i in range(3):
6     for j in range(i):
7         print(' ', end='')
8     for k in range(6-2*i, 0, -1): # 第二個參數填 0, 答案 (C)
9         print('*', end='')
10    print('')
```

### 3.4 二維陣列巢狀迴圈 (題目 18)

計算 5×5 迷宮中，內部 3×3 格每格的四方向鄰居中值為 1 的總數：

```

1 maze = [[1,1,1,1,1],
2         [1,0,1,0,1],
3         [1,1,0,0,1],
4         [1,0,0,1,1],
5         [1,1,1,1,1]]
6 count = 0
7 for i in range(1, 4): # 內部 row
8     for j in range(1, 4): # 內部 col
9         dir = [[-1,0],[0,1],[1,0],[0,-1]] # 上右下左
10        for d in range(4):
11            if maze[i+dir[d][0]][j+dir[d][1]] == 1:
12                count += 1
13 print(count) # 答案 20, 選 (B)
```



## 4 函式與遞迴

### 4.1 函式基本概念

```

1 def add(a, b):
2     return a + b          # 遇到 return 立刻結束函式
3
4 result = add(3, 5)        # result = 8

```

#### 核心觀念

函式執行到 `return` 就結束，後面的程式碼不會執行。  
 若函式沒有 `return`，或 `return` 後面沒有值，回傳 `None`。

### 4.2 遞迴的兩個必要條件

1. 終止條件 (base case)：讓遞迴能停下來的判斷
2. 縮減問題規模：每次呼叫必須讓問題往終止條件靠近

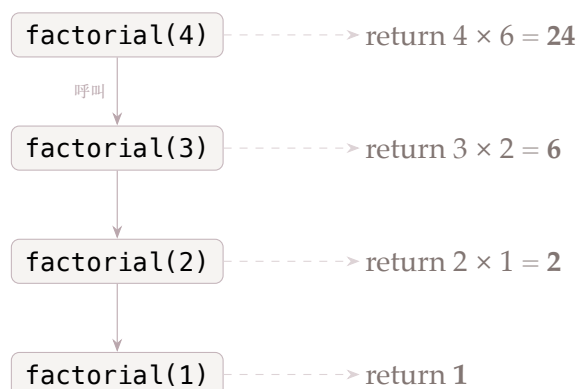
```

1 def factorial(n):
2     if n <= 1:                # 終止條件
3         return 1
4     return n * factorial(n - 1) # 縮減: n 每次減 1
5
6 # factorial(5) = 5*4*3*2*1 = 120

```

題目 11：計算  $n$  階乘的函式有 bug，把第 3 行 `if n >= 0` 改為 `if n > 0` 即可正確終止，答案 (B)。

*factorial(4)* 呼叫展開樹



#### 習題—遞迴追蹤

給定以下函式，請問 `f(5)` 的回傳值為何？寫出展開過程。

```

1 def f(n):

```

```

2     if n <= 0:
3         return 0
4     return n + f(n - 2)

```

### 4.3 遞迴追蹤的方法

1. 先找終止條件，知道哪些輸入直接回傳
2. 從最小的 case 往上推，建立一張展開表
3. 注意回傳後是否還有計算（`return f(n-1) + n` 中，`f(n-1)` 先算，回來才加 `n`）

題目 1： `f(10)` 展開

```

1 def f(i):
2     if i > 0:
3         if (i // 2) % 2 == 0:
4             return f(i-2) * i
5         return f(i-2) * (-i)
6     else:
7         return 1

```

| 呼叫                 | 條件 ( <code>(i//2)%2</code> )                  | 計算式                 | 回傳值   |
|--------------------|-----------------------------------------------|---------------------|-------|
| <code>f(0)</code>  | <code>i &lt;= 0</code> ，終止條件                  | —                   | 1     |
| <code>f(2)</code>  | <code>1%2 = 1 ≠ 0</code> ，乘 <code>(-i)</code> | $f(0) \times (-2)$  | -2    |
| <code>f(4)</code>  | <code>2%2 = 0</code> ，乘 <code>i</code>        | $f(2) \times 4$     | -8    |
| <code>f(6)</code>  | <code>3%2 = 1 ≠ 0</code> ，乘 <code>(-i)</code> | $f(4) \times (-6)$  | 48    |
| <code>f(8)</code>  | <code>4%2 = 0</code> ，乘 <code>i</code>        | $f(6) \times 8$     | 384   |
| <code>f(10)</code> | <code>5%2 = 1 ≠ 0</code> ，乘 <code>(-i)</code> | $f(8) \times (-10)$ | -3840 |

### 4.4 費式數列（題目 7、14）

```

1 # 遞迴版
2 def fib(n):
3     if n < 2:
4         return n
5     return fib(n-1) + fib(n-2)
6
7 # 迴圈版（題目 7 的結構）
8 a, b = 0, 1
9 for i in range(2, N+1):
10     temp = b
11     b = a + b    # (a) 填 b = a+b
12     a = temp
13 print(a)        # (b) 填 a

```

## 考試提示

迴圈版費式數列中，`temp = b` 的作用是暫存舊的 `b` 值，避免 `b` 更新後再去算 `a = temp` 時拿到錯誤的值。

用 Python 同時賦值可簡化為 `a, b = b, a + b`。

題目 14：`Mystery(9) = 34`，終止條件為 `x <= 1`，推出 `else` 的算式是 `Mystery(x-2) + Mystery(x-1)`，答案 (D)。

## 4.5 GCD 輾轉除法（題目 9、23）

## 核心觀念

輾轉除法原理： $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$

每次把「余數」替換「較大數」，直到余數為 0，此時「較小數」就是 GCD。

```

1 # 迴圈版（題目 9：填入 while 迴圈內容）
2 i, j = 76, 48
3 while (i % j) != 0:
4     k = i % j      # 保存余數
5     i = j
6     j = k          # (A) 正確答案
7 print(j)          # 印出 GCD = 4
8
9 # 遞迴版（題目 23：填入兩個 return）
10 def GCD(a, b):
11     r = a % b
12     if r == 0:
13         return b    # 填 b
14     return GCD(b, r) # 填 GCD(b, r)，答案 (B)

```

## 4.6 互相遞迴（題目 8、13）

兩函式互相呼叫時，追蹤時要記錄每次呼叫後要回來做什麼（類似堆疊）。

```

1 def f1(m):
2     if m > 3:
3         print(m); return
4     else:
5         print(m)
6         f2(m + 2)
7         print(m)      # f2 回來後才執行這行
8
9 def f2(n):
10     if n > 3:
11         print(n); return

```

```

12     else:
13         print(n)
14         f1(n - 1)
15         print(n)
16
17 f1(1)
18 # 執行順序:
19 #   f1(1) -> 印 1 -> f2(3) -> 印 3 -> f1(2) -> 印 2
20 #           -> f2(4) -> 印 4 -> 回 -> 印 2
21 #           -> 回 f2(3) -> 印 3 -> 回 f1(1) -> 印 1
22 # 輸出: 1 3 2 4 2 3 1

```

題目 8 問「以下敘述何者為錯」。追蹤呼叫路徑：

- f1(1) 呼叫 f2(3)，f2(3) 呼叫 f1(2)，f1(2) 呼叫 f2(4)
- f2 被呼叫的位置：f2(3)、f2(4)，共 2 次

選項 (C)「f2 一共被呼叫三次」是錯誤的，答案 (C)。

#### 4.7 遞迴冪次 (題目 21)

```

1 def G(a, x):
2     if x == 0:
3         return 1
4     else:
5         return a * G(a, x - 1)
6
7 # G(a, x) = a^x
8 # G(3, 7) = 3^7 = 2187, 答案 (B)

```

## 5 串列 (List)

### 5.1 基本操作

```

1 a = [1, 3, 5, 7, 9]
2 print(a[0])           # 1 (索引從 0 開始)
3 print(a[-1])          # 9 (負索引從末尾算)
4 print(len(a))         # 5
5
6 a.append(11)          # 末尾加入
7 a.sort()              # 原地排序 (升序)
8 b = sorted(a, reverse=True, key=lambda x: x) # 回傳新串列

```

串列索引示意

|     |    |    |    |    |    |
|-----|----|----|----|----|----|
| 正索引 | 0  | 1  | 2  | 3  | 4  |
|     | 1  | 3  | 5  | 7  | 9  |
| 負索引 | -5 | -4 | -3 | -2 | -1 |

### 習題—串列索引

`a = [10, 20, 30, 40, 50]`，請問以下各式的值為何？

- (a) `a[2]`
- (b) `a[-1]`
- (c) `a[-3]`
- (d) `len(a) - 1` (最後一個元素的正索引)

## 5.2 串列初始化

```

1 a = [0] * 10           # [0, 0, ..., 0] (10 個 0)
2 b = [i for i in range(101)] # [0, 1, 2, ..., 100] ← list
  ↳ comprehension
3 c = [i**2 for i in range(5)] # [0, 1, 4, 9, 16]
```

### 題目 30：前綴和

`b = [i for i in range(101)]` 即 `b[i] = i`。

接著 `a[i] = b[i] + a[i-1] = i + a[i-1]`，`a` 就是從 1 加到 `i` 的前綴和。

`a[50] - a[30] = (1+...+50) - (1+...+30) = 31+32+...+50 = 810`，答案 (D)。

## 5.3 同時賦值 (題目 15)

Python 的同時賦值會先把右側全部算出來，再一次性賦值：

```

1 a, b = b, a           # 交換兩個變數，不需要 temp
2
3 # 題目 15:
4 a = [1, 3, 5, 7, 9, 8, 6, 4, 2]
5 n = 9
6 for i in range(n):
7     a[i], a[n-i-1] = a[n-i-1], a[i] # 同時交換，反轉陣列
8 # 反轉後: [2, 4, 6, 8, 9, 7, 5, 3, 1]
9 # 但每個 i 都做, i=0 時換 a[0]↔a[8], i=4 時換 a[4]↔a[4].....
10 # 最終 a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 9] (含重複邊界)
11 # 印出第一個 n//2+1 = 5 對 → 1 2 3 4 5 6 7 8 9 9, 答案 (C)
```

## 5.4 索引偏移 (題目 25)

```

1 X = [0] * 10
```

```

2 for i in range(10):
3     X[(i+2) % 10] = int(input())    # 輸入 0,1,...,9
4
5 # i=0 → X[2]=0; i=1 → X[3]=1; ...; i=7 → X[9]=7
6 # i=8 → X[0]=8; i=9 → X[1]=9
7 # 結果: X = [8, 9, 0, 1, 2, 3, 4, 5, 6, 7], 答案 (D)

```

## 5.5 搜尋演算法 (題目 6、22、29)

### 線性搜尋

```

1 def linear_search(a, value):
2     for i in range(len(a)):        # 逐一比較
3         if a[i] == value:
4             return i
5     return -1

```

100 個元素，找 `a[33]=100`：迴圈執行到 `i=33` 時找到，`while` 主體共執行 34 次 ( $n1 = 34$ )。

### 二分搜尋

```

1 def binary_search(a, value):
2     low, high = 0, len(a) - 1
3     while low <= high:
4         mid = (low + high) // 2
5         if a[mid] == value:
6             return mid
7         elif a[mid] < value:
8             low = mid + 1
9         else:
10            high = mid - 1
11    return -1

```

#### 核心觀念

二分搜尋每次縮減一半，100 個元素最多需  $\lceil \log_2 100 \rceil = 7$  次，本題實際追蹤為 5 次 ( $n2 = 5$ )。前提：陣列必須已排序。

題目 22 (二分搜尋有 bug)：函式目的是找「A 中大於 x 的最小值」。當 `x=16` (已超出 A 最大值 14)，`high` 最終指向 index 7，`A[7]=14` 並非大於 16，程式回傳錯誤，答案 (D)。

## 5.6 最大值、最小值追蹤 (題目 24、29)

```

1 p = q = A[0]    # 同時初始化為第一個元素
2 for i in range(1, n):
3     if A[i] > p:

```

```

4     p = A[i]          # p 追最大值
5     if A[i] < q:
6         q = A[i]      # q 追最小值

```

### 題目 24：「不一定正確」的陷阱

若陣列所有元素相同，則 `p == q`，「`q < p`」為 False。  
 選項 (C) `q < p` 不一定正確，為答案。

題目 29：程式初始 `M = N = s = -1, 101, 3`，只有 3 個元素，`elif A[i] < N` 用 `elif` 而非 `if`，當 `A[i] > M` 成立時跳過最小值判斷，找測資讓 `A[0] < A[1] < A[2]` 才能觸發錯誤，答案 `[80, 90, 100]`，選 (B)。

## 6 程式識讀技巧

### 6.1 執行流程追蹤四步驟

1. 找起點：從 top-level 程式或函式呼叫點開始
2. 建變數表：每個變數一欄，逐行更新
3. 展開迴圈/遞迴：把每輪的值逐一記錄，不要跳步
4. 確認輸出時機：`print` 在迴圈裡還是外？遞迴回來後還有沒有輸出？

### 6.2 死程式碼 (Dead Code，題目 27)

「永遠執行不到」的程式碼：

```

1 def F(a):
2     while a < 10:
3         a = a + 5
4         # while 結束後: a 只可能是 10 (0+5+5) 或 15、20...
5         if a < 12:
6             a = a + 2      # 若 a=10 → a 變 12
7         if a < 11:
8             a = 5          # a 只可能是 12, 15, 20..., 永遠 >= 11, 這行執行不到

```

答案 (C)：`a = 5` 永遠不會執行。

### 6.3 冗餘程式碼 (題目 20)

「冗餘」= 移除後程式行為不變。

判斷策略：某段程式計算的值，在後面馬上就被覆蓋或根本不用。

`Change % 1` 的結果永遠是 0 (任何整數除以 1 餘 0)，D 區的 `Change = Change % 1` 把 `Change` 清零，但清完之後已經沒有後續使用，而 D 區的 `print` 只印出「1 元硬幣」的張數，`Change = Change % 1` 這行是冗餘的，答案 (D)。

## 6.4 填空題策略（題目 2、3、9、12、16、19、23）

1. 逆向推導：已知輸出，從輸出往回推條件或算式
2. 代入選項：把四個選項逐一代入，看哪個產生正確結果
3. 邊界驗證：除了正常情況，也試試最小值或最大值

## 7 程式實作題攻略

---

### 7.1 解題流程

1. 讀清楚輸入格式：幾行？每行幾個數？用什麼隔開？
2. 讀清楚輸出格式：空格？換行？特殊情況（如 `None`）？
3. 先對範例：用題目的範例輸入手動跑一遍，確認理解題意
4. 實作並測試：先過範例，再想邊界情況

### 7.2 實作題型一：七言對聯（2021/9）



## 七言對聯程式實作 2021 年 9 月

### 問題描述

每首七言對聯有兩句，每句七個字，每字的平仄以 0（平聲）或 1（仄聲）表示。請檢查對聯是否符合以下三條規則，並輸出所有違反的規則編號：

- 規則 A（二四不同二六同）：每句第 2 字與第 4 字的平仄必須不同；第 2 字與第 6 字的平仄必須相同。（兩句都要檢查）
- 規則 B（仄起平收）：第一句第 7 字必須是仄聲，第二句第 7 字必須是平聲。
- 規則 C（同聯相對）：第一句的第 2、4、6 字的平仄，必須分別與第二句的第 2、4、6 字的平仄相反。

### 輸入格式

第一行為正整數  $n$  ( $1 \leq n \leq 30$ )，表示有幾首對聯。接下來每 2 行為一首對聯，每行 7 個數字（0 或 1），以空白間隔。

### 輸出格式

輸出  $n$  行。每行列出該對聯違反的規則字母（依字母順序，無空白）。若全部符合則輸出 `None`。

### 範例

| 輸入            | 輸出 |
|---------------|----|
| 1             | AC |
| 1 0 0 0 0 1 1 |    |
| 0 1 0 0 1 1 0 |    |

知識點：陣列索引比對、多條件判斷、集合去重、排序輸出

```

1 n = int(input())
2 for _ in range(n):
3     r1 = list(map(int, input().split()))
4     r2 = list(map(int, input().split()))
5     bad = set()
6
7     # 規則 A: 二四不同二六同 (索引 1,3,5, 從 0 開始)
8     if r1[1] == r1[3] or r1[1] != r1[5]:
9         bad.add('A')
10    if r2[1] == r2[3] or r2[1] != r2[5]:
11        bad.add('A')
12
13    # 規則 B: 仄起平收 (第 7 字 = 索引 6; 仄 =1, 平 =0)
14    if r1[6] != 1 or r2[6] != 0:
15        bad.add('B')
16
17    # 規則 C: 同聯相對 (第 2,4,6 字, 索引 1,3,5 必須互反)

```

```

18     for idx in [1, 3, 5]:
19         if r1[idx] == r2[idx]:
20             bad.add('C')
21
22     print(''.join(sorted(bad)) if bad else 'None')

```

### 7.3 實作題型二：裝飲料 (2024/10)

#### 裝飲料 程式實作 2024 年 10 月

##### 問題描述

一個冷飲杯的內層空間由上下兩個長方體相接組成：下方底面為  $w_1 \times w_1 \text{ cm}^2$ ，高  $h_1 \text{ cm}$ ；上方底面為  $w_2 \times w_2 \text{ cm}^2$ ，高  $h_2 \text{ cm}$  ( $w_1 < w_2$ )。

已知依序倒入  $N$  杯冰水，體積分別為  $V_1, V_2, \dots, V_N$  (單位 ml,  $1 \text{ cm}^3 = 1 \text{ ml}$ )。求每次加水後水位上升高度的最大值。杯子裝滿後水位不再上升 (上升值為 0)。每次上升高度保證為整數。

##### 輸入格式

第一行正整數  $N$  ( $1 \leq N \leq 10$ )。第二行四個正整數  $w_1, w_2, h_1, h_2$  ( $1 \leq w_1 < w_2, h_1, h_2 \leq 50$ )。第三行  $N$  個正整數，每次加水體積不超過  $10^6$ 。

##### 輸出格式

一個整數，為水位上升高度的最大值。

##### 範例

| 輸入        | 輸出 |
|-----------|----|
| 2         | 13 |
| 5 10 12 8 |    |
| 400 600   |    |

說明：第一杯 400 ml 裝入後水升 13 cm，第二杯 600 ml 裝入後水升 6 cm，最大值為 13。

知識點：分段體積計算 (下段/上段)、加水後高度、最大值追蹤

```

1 N = int(input())
2 w1, w2, h1, h2 = map(int, input().split())
3 vols = list(map(int, input().split()))
4
5 cap1 = w1 * w1 * h1      # 下段總容量
6 cap2 = w2 * w2 * h2      # 上段總容量
7
8 cur_h = 0
9 max_rise = 0
10

```

```

11 for v in vols:
12     # 計算目前水量
13     if cur_h <= h1:
14         cur_v = cur_h * w1 * w1
15     else:
16         cur_v = cap1 + (cur_h - h1) * w2 * w2
17
18     new_v = min(cur_v + v, cap1 + cap2) # 不超過總容量
19
20     # 計算新高度
21     if new_v <= cap1:
22         new_h = new_v // (w1 * w1)
23     else:
24         new_h = h1 + (new_v - cap1) // (w2 * w2)
25
26     rise = new_h - cur_h
27     max_rise = max(max_rise, rise)
28     cur_h = new_h
29
30 print(max_rise)

```

## 7.4 實作題型三：遊戲選角 (2024/1)

### 遊戲選角 程式實作 2024 年 1 月

#### 問題描述

遊戲中每個角色有攻擊力和防禦力兩個屬性，綜合能力定義為：

$$\text{綜合能力} = \text{攻擊力}^2 + \text{防禦力}^2$$

給定  $n$  名角色的屬性，找出綜合能力第二高的角色，輸出其攻擊力與防禦力。題目保證所有角色的綜合能力皆相異。

#### 輸入格式

第一行整數  $n$  ( $3 \leq n \leq 20$ )。接下來  $n$  行，每行兩個整數，分別為攻擊力和防禦力 ( $1 \leq \text{各值} \leq 100$ )。

#### 輸出格式

一行，兩個整數，依序為綜合能力第二高角色的攻擊力與防禦力，以空白間隔。

#### 範例

| 輸入  | 輸出  |
|-----|-----|
| 3   | 1 4 |
| 3 2 |     |
| 5 2 |     |
| 1 4 |     |

說明：三名角色的綜合能力分別為 13、29、17，第二高（17）為第三位角色，輸出 14。

知識點：自定義排序 key（`lambda`）、找第二高值

```
1 n = int(input())
2 chars = []
3 for _ in range(n):
4     a, d = map(int, input().split())
5     chars.append((a, d, a**2 + d**2))
6
7 # 依綜合能力降序排列
8 chars.sort(key=lambda x: x[2], reverse=True)
9
10 # 題目保證所有角色綜合能力相異，第二高即 index = 1
11 print(chars[1][0], chars[1][1])
```

#### 考試提示

- `sorted()` 回傳新串列，原串列不變
- `list.sort()` 原地排序，回傳 `None`
- 兩者都可接受 `key=` 和 `reverse=True`

## 8 程式識讀完整題庫

### 題目 1

給定右側函式 `f()`，當執行 `f(10)` 時，最終回傳結果為何？

```
1 def f(i):  
2     if i > 0:  
3         if (i // 2) % 2 == 0:  
4             return f(i - 2) * i  
5         return f(i - 2) * (-i)  
6     else:  
7         return 1
```

- (A) 1
- (B) 3840
- (C) -3840
- (D) 執行時導致無窮迴圈，不會停止執行

知識點：遞迴追蹤、整數除法 (`//`)、取餘 (`%`)

### 題目 2

給定右側程式，若已知輸出的結果為 `[1] [2] [3] [5] [4] [6]`，程式中的 (?) 應為下列何者？

```
1 for i in range(5):  
2     j = 0  
3     while (  
4         print(f'[{i} + {j}]', end = '  
5         j = j + 2
```

- (A) `j < i`
- (B) `j > i`
- (C) `j <= i`
- (D) `j >= i`

知識點：`while` 條件填空、`f-string` 輸出

## 題目 3

給定右側函式 `f()`，已知 `f(14)`、`f(10)`、`f(6)` 分別回傳 25、18、10，函式中的 (?) 應為下列何者？

```
1 def f(n):  
2     if n < 2:  
3         return n  
4     else:  
5         return n + f( (?) )
```

(A)  $(n+1)/2$

(B)  $n/2$

(C)  $(n-1)/2$

(D)  $(n/2)+1$

知識點：遞迴規律推導、整數除法

## 題目 4

給定右側程式片段，當程式執行完後，輸出結果為何？

```
1 Q = [0] * 200  
2 val = 0  
3 count = 0  
4 head = tail = 0  
5 for i in range(1, 31):  
6     Q[tail] = i  
7     tail = tail + 1  
8 while tail > head + 1:  
9     val = Q[head]  
10    head = head + 1  
11    count = count + 1  
12    if count == 3:  
13        count = 0  
14        Q[tail] = val  
15        tail = tail + 1  
16 print(Q[head])
```

(A) 9

(B) 18

(C) 27

(D) 30

知識點：串列模擬、*head/tail* 指標（佇列概念）

## 題目 5

右側程式正確的輸出應該如下：

```
      *
    * * *
  * * * * *
* * * * * * *
* * * * * * * *
```

在不修改右側程式之第 4 行及第 5 行程式碼的前提下，最少需修改幾行程式碼以得到正確輸出？

```
1 k = 4
2 m = 1
3 for i in range(5):
4     print(' ' * k, end = '')
5     print('*' * m)
6     k = k - 1
7     m = m + 1
```

- (A) 1
- (B) 2
- (C) 3
- (D) 4

知識點：迴圈結構、字串重複(\*運算子)

## 題目 6

給定一整數列表  $a[0]$ 、 $a[1]$ 、...、 $a[99]$  且  $a[k]=3k+1$ ，以  $value=100$  呼叫以下兩函式，假設函式  $f1$  及  $f2$  之 `while` 迴圈主體分別執行  $n1$  與  $n2$  次（也就是說計算 `if` 敘述執行次數，不包含 `elif` 敘述），請問  $n1$  與  $n2$  之值為何？

```
1 def f1(a, value):          def f2(a, value):
2     r_value = -1           r_value = -1
3     i = 0                  low = 0
4     while i < 100:         high = 99
5         n1 += 1            while low <= high:
6         if a[i] == value:  n2 += 1
7             r_value = i    mid = (low + high) // 2
8             break          if a[mid] == value:
9         i = i + 1          r_value = mid
10        return r_value     break
11                            elif a[mid] < value:
12                            low = mid + 1
13                            else:
14                            high = mid - 1
15                            return r_value
```

- (A)  $n1=33, n2=4$
- (B)  $n1=33, n2=5$
- (C)  $n1=34, n2=4$
- (D)  $n1=34, n2=5$

知識點：線性搜尋 vs 二分搜尋、迴圈次數計算



## 題目 7

一個費式數列定義第一個數為 0，第二個數為 1，之後的每個數都等於前兩個數相加，如下所示：0、1、1、2、3、5、8、13、21、34、55、89……右列的程式用以計算並印出第 N 個 ( $N \geq 2$ ) 費式數列的數值，請問 (a) 與 (b) 兩個空格的敘述 (statement) 應該為何？

```

1 a = 0
2 b = 1
3 for i in range(2, N + 1):
4     temp = b
5     (a)
6     a = temp
7 print( (b) )

```

- (A) (a)  $f[i] = f[i-1] + f[i-2]$  (b)  $f[N]$   
 (B) (a)  $a = a + b$  (b)  $a$   
 (C) (a)  $b = a + b$  (b)  $a$   
 (D) (a)  $f[i] = f[i-1] + f[i-2]$  (b)  $f[i]$

知識點：費式數列、暫存變數 (*temp*)、迴圈實作

## 題目 8

給定右側函式  $f1()$  及  $f2()$ 。  $f1(1)$  運算過程中，以下敘述何者為錯？

```

1 def f1(m):                def f2(n):
2     if m > 3:              if n > 3:
3         print(m)           print(n)
4         return             return
5     else:                  else:
6         print(m)           print(n)
7         f2(m + 2)          f1(n - 1)
8         print(m)           print(n)

```

- (A) 印出的數字最大的是 4  
 (B)  $f1$  一共被呼叫二次  
 (C)  $f2$  一共被呼叫三次  
 (D) 數字 2 被印出兩次

知識點：互相遞迴、執行流程追蹤、呼叫次數

## 題目 9

右側程式片段擬以輾轉除法求 `i` 與 `j` 的最大公因數。請問 `while` 迴圈內容何者正確？(注意：`(low + high) // 2` 只取整數部分)

```
1 i = 76
2 j = 48
3 while (i % j) != 0:
4     _____
5     _____
6     _____
7 print(j)
```

- (A) `k = i % j`  
`i = j`  
`j = k`
- (B) `i = j`  
`j = k`  
`k = i % j`
- (C) `i = j`  
`j = i % k`  
`k = i`
- (D) `k = i`  
`i = j`  
`j = i % k`

知識點：輾轉除法 (GCD)、`while` 迴圈填空

## 題目 10

右側程式執行過後所輸出數值為何？

```
1 count = 10
2 if count > 0:
3     count = 11
4 if count > 10:
5     count = 12
6     if count % 3 == 4:
7         count = 1
8     else:
9         count = 0
10 elif count > 11:
11     count = 13
12 else:
13     count = 14
14 if count:
15     count = 15
16 else:
17     count = 16
18 print(count)
```

- (A) 11
- (B) 13
- (C) 15
- (D) 16

知識點：獨立 *if* 追蹤、*if/elif* 執行順序、布林值判斷

## 題目 11

右側為一個計算  $n$  階乘的函式，請問該如何修改才會得到正確的結果？

```
1 def fun(n):  
2     fac = 1  
3     if n >= 0:  
4         fac = n * fun(n - 1)  
5     return fac
```

- (A) 第 2 行，改為 `fac = n`
- (B) 第 3 行，改為 `if n > 0:`
- (C) 第 4 行，改為 `fac = n * fun(n+1)`
- (D) 第 4 行，改為 `fac = fac * fun(n-1)`

知識點：遞迴終止條件 (*base case*)、階乘

## 題目 12

右側 `f()` 函式 (a)、(b)、(c) 處需分別填入哪些數字，方能使得 `f(4)` 輸出 2468 的結果？

```
1 def f(n):  
2     p = 0  
3     i = n  
4     while i >= (a):  
5         p = 10 - (b) * i  
6         print(p, end='')  
7         i = i - (c)
```

- (A) 1, 2, 1
- (B) 0, 1, 2
- (C) 0, 2, 1
- (D) 1, 1, 1

知識點：*while* 條件與迴圈體填空、輸出預測

## 題目 13

右側 `g(4)` 函式呼叫執行後，回傳值為何？

```
1 def f(n):
2     if n > 3:
3         return 1
4     elif n == 2:
5         return 3 + f(n + 1)
6     else:
7         return 1 + f(n + 1)
8
9 def g(n):
10     j = 0
11     for i in range(1, n):
12         j = j + f(i)
13     return j
```

- (A) 6
- (B) 11
- (C) 13
- (D) 14

知識點：遞迴 + `for` 迴圈混合追蹤、累加

## 題目 14

右側 `Mystery()` 函式 `else` 部分運算式應為何，才能使得 `Mystery(9)` 的回傳值為 34。

```
1 def Mystery(x):
2     if x <= 1:
3         return x
4     else:
5         return _____
```

- (A) `x + Mystery(x-1)`
- (B) `x * Mystery(x-1)`
- (C) `Mystery(x-2) + Mystery(x+2)`
- (D) `Mystery(x-2) + Mystery(x-1)`

知識點：遞迴終止條件推導、逆向推算

## 題目 15

右側程式碼執行後，輸出結果為何？

```
1 a = [1, 3, 5, 7, 9, 8, 6, 4, 2]
2 n = 9
3 for i in range(n):
4     a[i], a[n-i-1] = a[n-i-1], a[i]
5 for i in range(n//2 + 1):
6     print(a[i], a[n-i-1], end=' ')
```

- (A) 2468975319
- (B) 1357924689
- (C) 1234567899
- (D) 2468513799

知識點：串列同時賦值、反轉、索引計算

## 題目 16

右側函式以 `F(7)` 呼叫後回傳值為 12，則 `<condition>` 應為何？

```
1 def F(a):
2     if <condition>:
3         return 1
4     else:
5         return F(a-2) + F(a-3)
```

- (A)  $a < 3$
- (B)  $a < 2$
- (C)  $a < 1$
- (D)  $a < 0$

知識點：遞迴終止條件填空、逆向推算

## 題目 17

右側是依據分數 `s` 評定等第的程式碼片段，正確的等第公式應為：90~100 判為 A 等、80~89 判為 B 等、70~79 判為 C 等、60~69 判為 D 等、0~59 判為 F 等。這段程式碼在處理 0~100 的分數時，有幾個分數的等第是錯的？

```
1 if s >= 90:
2     print('A')
3 elif s >= 80:
4     print('B')
5 elif s > 60:
6     print('D')
7 elif s > 70:
8     print('C')
9 else:
10    print('F')
```

- (A) 20
- (B) 11
- (C) 2
- (D) 10

知識點：if/elif 條件順序、邊界值 ( $>$  vs  $\geq$ )、錯誤計數

## 題目 18

右側程式片段執行後，`count` 的值為何？

```

1 maze = [[1, 1, 1, 1, 1],
2          [1, 0, 1, 0, 1],
3          [1, 1, 0, 0, 1],
4          [1, 0, 0, 1, 1],
5          [1, 1, 1, 1, 1]]
6 count = 0
7 for i in range(1, 4):
8     for j in range(1, 4):
9         dir = [[-1,0], [0,1], [1,0], [0,-1]]
10        for d in range(4):
11            if maze[i+dir[d][0]][j+dir[d][1]] == 1:
12                count = count + 1

```

- (A) 36
- (B) 20
- (C) 12
- (D) 3

知識點：二維串列、巢狀迴圈、方向向量

## 題目 19

右側程式片段中執行後若要印出下列圖案，`(a)` 的條件判斷式該如何設定？

```

* * * * *
  * * * *
    * *

```

```

1 for i in range(4):
2     for j in range(i):
3         print(' ', end='')
4     for k in range(6-2*i, (a), -1):
5         print('*', end='')
6     print('')

```

- (A) 2
- (B) 1
- (C) 0
- (D) -1

知識點：`range` 參數填空、倒三角圖形



## 題目 20

下列程式碼是自動計算找零程式的一部分，程式碼中三個主要變數分別為 `Total`（購買總額）、`Paid`（實際支付金額）、`Change`（找零金額）。但是此程式片段有冗餘的程式碼，也就是移除該段程式碼之後不會影響程式的功能。請找出冗餘程式碼的區塊。

```
1 Change = Paid - Total
2 print(f'500 : {(Change-Change%500)//500} pieces')
3 Change = Change % 500
4 print(f'100 : {(Change-Change%100)//100} coins')
5 Change = Change % 100
6 # A 區
7 print(f'50 : {(Change-Change%50)//50} coins')
8 Change = Change % 50
9 # B 區
10 print(f'10 : {(Change-Change%10)//10} coins')
11 Change = Change % 10
12 # C 區
13 print(f'5 : {(Change-Change%5)//5} coins')
14 Change = Change % 5
15 # D 區
16 print(f'1 : {(Change-Change%1)//1} coins')
17 Change = Change % 1
```

- (A) 冗餘程式碼在 A 區
- (B) 冗餘程式碼在 B 區
- (C) 冗餘程式碼在 C 區
- (D) 冗餘程式碼在 D 區

知識點：冗餘程式碼識別（% 1 的性質）

## 題目 21

右側 `G()` 為遞迴函式，`G(3, 7)` 執行後回傳值為何？

```
1 def G(a, x):  
2     if x == 0:  
3         return 1  
4     else:  
5         return a * G(a, x - 1)
```

- (A) 128
- (B) 2187
- (C) 6561
- (D) 1024

知識點：遞迴冪次、 $G(a, x) = a^x$

## 題目 22

給定一個  $1 \times 8$  的列表 `A`，`A = [0, 2, 4, 6, 8, 10, 12, 14]`。右側函式 `Search(x)` 真正目的是找到 `A` 之中大於 `x` 的最小值。然而，這個函式有誤。請問下列哪個函式呼叫可以測出函式有誤？

```
1 A = [0, 2, 4, 6, 8, 10, 12, 14]  
2 def Search(x):  
3     high = 7  
4     low = 0  
5     while high > low:  
6         mid = (high + low) // 2  
7         if A[mid] <= x:  
8             low = mid + 1  
9         else:  
10            high = mid  
11     return A[high]
```

- (A) `Search(-1)`
- (B) `Search(0)`
- (C) `Search(10)`
- (D) `Search(16)`

知識點：二分搜尋邏輯、邊界測試（錯誤觸發）

## 題目 23

右側函式兩個回傳式分別該如何撰寫，才能正確計算並回傳參數 `a`、`b` 之最大公因數 (Greatest Common Divisor)？

```
1 def GCD(a, b):  
2     r = a % b  
3     if r == 0:  
4         return ____  
5     return ____
```

- (A) `a, GCD(b,r)`
- (B) `b, GCD(b,r)`
- (C) `a, GCD(a,r)`
- (D) `b, GCD(a,r)`

知識點：GCD 遞迴填空、輾轉除法

## 題目 24

若 `A` 是一個可儲存 `n` 筆整數的列表，且資料儲存於 `A[0] ~ A[n-1]`。經過右側程式碼運算後，以下何者敘述不一定正確？

```
1 A = [...]  
2 p = q = A[0]  
3 for i in range(1, n):  
4     if A[i] > p:  
5         p = A[i]  
6     if A[i] < q:  
7         q = A[i]
```

- (A) `p` 是 `A` 列表資料中的最大值
- (B) `q` 是 `A` 列表資料中的最小值
- (C) `q < p`
- (D) `A[0] ≤ p`

知識點：最大最小值追蹤、「不一定正確」邏輯陷阱

## 題目 25

右側 `F()` 函式執行時，若輸入依序為整數 0, 1, 2, 3, 4, 5, 6, 7, 8, 9，請問 `X` 列表的元素值依序為何？

```
1 def F():  
2     X = [0] * 10  
3     for i in range(10):  
4         X[(i+2)%10] = int(input())
```

- (A) 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- (B) 2, 0, 2, 0, 2, 0, 2, 0, 2, 0
- (C) 9, 0, 1, 2, 3, 4, 5, 6, 7, 8
- (D) 8, 9, 0, 1, 2, 3, 4, 5, 6, 7

知識點：串列索引偏移  $((i+2)\%10)$ 、模運算

## 題目 26

右側程式片段無法正確列印 20 次的 `"Hi!"`，請問下列哪一個修改方式仍無法正確列印 20 次的 `"Hi!"`？

```
1 for i in range(0, 101, 5):  
2     print('Hi!')
```

- (A) 將 `range(0,101,5)` 改為 `range(0,20,1)`
- (B) 將 `range(0,101,5)` 改為 `range(5,101,5)`
- (C) 將 `range(0,101,5)` 改為 `range(0,100,5)`
- (D) 將 `range(0,101,5)` 改為 `range(5,100,5)`

知識點：`range` 元素數量、*off-by-one*

## 題目 27

給定右側函式 `F()`，執行 `F()` 時哪一行程式碼永遠不會被執行到？

```
1 def F(a):  
2     while a < 10:  
3         a = a + 5  
4         if a < 12:  
5             a = a + 2  
6         if a < 11:  
7             a = 5
```

- (A) `a = a + 5`
- (B) `a = a + 2`
- (C) `a = 5`
- (D) 每一行都執行得到

知識點：while 後變數範圍、dead code 分析

## 題目 28

給定右側函式 `F()`，`F()` 執行完所回傳的 `x` 值為何？

```
1 def F(n):  
2     x = 0  
3     for i in range(1, n+1):  
4         for j in range(i, n+1):  
5             k = 1  
6             while k <= n:  
7                 x = x + 1  
8                 k = k * 2  
9     return x
```

- (A)  $n(n+1)\sqrt{\lceil \log_2 n \rceil}$
- (B)  $n^2(n+1)/2$
- (C)  $n(n+1)(\lceil \log_2 n \rceil + 1)/2$
- (D)  $n(n+1)/2$

知識點：三層巢狀迴圈、複雜度 ( $\log$  因子)

## 題目 29

右側程式擬找出列表 A 中的最大值 (M) 和最小值 (N)。不過，這段程式碼有誤，請問 A 初始值如何設定就可以測出程式有誤？

```
1 M, N, s = -1, 101, 3
2 A = [80, 90, 100]
3 for i in range(s):
4     if A[i] > M:
5         M = A[i]
6     elif A[i] < N:
7         N = A[i]
8 print(f'M = {M}, N = {N}')
```

- (A) [90, 80, 100]
- (B) [80, 90, 100]
- (C) [100, 90, 80]
- (D) [90, 100, 80]

知識點: *elif* 邏輯漏洞、最大最小值、錯誤測資設計

## 題目 30

經過運算後，右側程式的輸出為何？

```
1 a = [0] * 101
2 b = [i for i in range(101)]
3 for i in range(1, 101):
4     a[i] = b[i] + a[i - 1]
5 print(a[50] - a[30])
```

- (A) 1275
- (B) 20
- (C) 1000
- (D) 810

知識點: *list comprehension*、前綴和 (*prefix sum*)

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 題號 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 答案 | C  | A  | B  | B  | A  | D  | C  | C  | A  | D  |
| 題號 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 答案 | B  | A  | C  | D  | C  | D  | B  | B  | C  | D  |
| 題號 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
| 答案 | B  | D  | B  | C  | D  | D  | C  | C  | B  | D  |

## 9 考前快速記憶表

### 9.1 必背語法

| 語法                                          | 說明                 |
|---------------------------------------------|--------------------|
| <code>a // b</code>                         | 整數除法（向下取整）         |
| <code>a % b</code>                          | 取餘數                |
| <code>range(s, e, step)</code>              | 等差數列，含 s 不含 e      |
| <code>a, b = b, a</code>                    | 同時賦值交換，右側全部先算      |
| <code>[0] * n</code>                        | 長度 n 的全零串列         |
| <code>[x for x in L]</code>                 | list comprehension |
| <code>sorted(L, key=f, reverse=True)</code> | 排序，回傳新串列           |
| <code>f'{var}'</code>                       | f-string 格式化輸出     |
| <code>set.add(x)</code>                     | 集合加入元素（自動去重）       |
| <code>''.join(sorted(s))</code>             | 集合 / 列表排序後串接成字串    |

### 9.2 識讀題分類速查

| 題型    | 對應題號                    | 核心技能               |
|-------|-------------------------|--------------------|
| 遞迴追蹤  | 1, 3, 8, 13, 14, 21, 23 | 展開表、返回值累積、互相遞迴     |
| 迴圈控制  | 2, 5, 7, 19, 26, 27     | range 參數、更新條件、死程式碼 |
| 條件判斷  | 10, 11, 17, 20          | 執行順序、覆蓋效果、冗餘程式碼    |
| 陣列操作  | 4, 15, 18, 24, 25, 30   | 索引偏移、同時賦值、二維陣列     |
| 搜尋演算法 | 6, 22, 29               | 線性 / 二分、次數計算、錯誤測資  |
| 填空題   | 2, 3, 9, 12, 16, 19, 23 | 逆向推導、代入驗證          |

### 9.3 實作題得分策略

1. 直接寫完整解：初級題的  $n=1$  與一般  $n$  用的是同一份程式碼，迴圈本來就能處理任何筆數，不需要特別為簡化情況另寫一版。
2. 先確認讀取正確：在主邏輯之前先把輸入印出來，確認格式沒讀錯再繼續。
3. 邊界情況要測：最小輸入（ $n=1$ ）、容量滿（水位不再上升）、排序後最大最小值是否正確。
4. 輸出格式最後檢查：空格、換行、特殊情況（如 `None`）都要對照題目再確認一次。



## 練習題解答

習題—條件追蹤 逐行追蹤：

| 步驟                          | 執行的語句                  | x 的值 |
|-----------------------------|------------------------|------|
| 初始                          | <code>x = 5</code>     | 5    |
| <code>if x &gt; 3</code> 成立 | <code>x = x + 2</code> | 7    |
| <code>if x &gt; 6</code> 成立 | <code>x = x * 2</code> | 14   |
| ( <code>elif</code> 被跳過)    | —                      | 14   |

答案：`print(x)` 輸出 **14**。

習題—`range` 計算

- (a) `range(3, 12)`：元素為 3, 4, ..., 11，共  $12 - 3 = 9$  個。
- (b) `range(0, 20, 3)`：元素為 0, 3, 6, 9, 12, 15, 18，共  $\lfloor (20 - 0 - 1) / 3 \rfloor + 1 = 7$  個。
- (c) `range(10, 1, -2)`：元素為 10, 8, 6, 4, 2，共 5 個。

習題—遞迴追蹤 `f(5)` 的展開：

| 呼叫                 | 結果                                      |
|--------------------|-----------------------------------------|
| <code>f(5)</code>  | $5 + f(3)$                              |
| <code>f(3)</code>  | $3 + f(1)$                              |
| <code>f(1)</code>  | $1 + f(-1)$                             |
| <code>f(-1)</code> | 0 (終止條件 $n \leq 0$ )                    |
| 回推                 | $1 + 0 = 1$ ; $3 + 1 = 4$ ; $5 + 4 = 9$ |

答案：`f(5)` 回傳 **9**。

習題—串列索引 `a = [10, 20, 30, 40, 50]`

- (a) `a[2]` = 30
- (b) `a[-1]` = 50 (最後一個)
- (c) `a[-3]` = 30 (倒數第三個)
- (d) `len(a) - 1` =  $5 - 1 = 4$